



字符串算法

字符串算法是noip考试以及省选noi考试中的一个重要模块，虽然最近几年考查次数比较少，但是其中设计的算法，比如字符串匹配，回文串，最长公共前缀，最长公共后缀等等是非常重要的。



字符读入输出

在涉及到字符以及字符串读入的时候，已经要注意格式，因为“空格”“回车”都属于字符，所以一定要去除一些无关操作的影响。

(1) 数字 字符

```
scanf("%d %c", &a, &b);
```

(2) 第一行一个数字，后面n行，每行一个字符+数字

```
scanf("%d", &n);  
for(int i=1; i<=n; i++)  
scanf("\n%c %d", &a, &b);
```

如果空格或者回车之后是数字的，就不需要考虑，因为空格和回车不是数字，是不会误判的，当然你也可以用%s读入一个字符，这样也不需要考虑空格了。

判断是否读入错误的办法很简单，把你的输入输出一遍就可以了，比如%d...%c...\n的形式。

字符串hash

当我们需要比较两个数字的时候，每次比较花费的时间是 $O(1)$ 的，但是我们如果比较两个字符串，每次比较花费的时间确实 $O(len)$ 的，即逐个字符比较，这样比较效率是很低的，所以我们可以考虑把字符串转换成数字来比较，也就是字符串hash，在设计hash函数的时候一定要注意解决冲突，即两个不同的字符串的hash值可能一样。目前一般采用数组挂链的形式，对于map判重，不建议使用，除非时间上来不及或者实在是没办法，否则最好不使用，因为效率太低了，虽然理论上时间复杂度为 $n \log n$ ，但是常数特别大，NOIP一般都会被卡。

把一个字符串str转换成hash值：

```
for(int i=0;i<l;i++)  
sum=(sum*131+str[i])%1000007;
```

字符串hash

当我们需要比较两个数字的时候，每次比较花费的时间是 $O(1)$ 的，但是我们如果比较两个字符串，每次比较花费的时间确实 $O(len)$ 的，即逐个字符比较，这样比较效率是很低的，所以我们可以考虑把字符串转换成数字来比较，也就是字符串hash，在设计hash函数的时候一定要注意解决冲突，即两个不同的字符串的hash值可能一样。目前一般采用数组挂链的形式，对于map判重，不建议使用，除非时间上来不及或者实在是没办法，否则最好不使用，因为效率太低了，虽然理论上时间复杂度为 $n \log n$ ，但是常数特别大，NOIP一般都会被卡。

把一个字符串str转换成hash值：

```
for(int i=0;i<l;i++)  
sum=(sum*131+str[i])%1000007;
```

是否要模一个质数取决于是否需要用桶来存储。

Hash算法

```
void hash(char *s, int k)
{
    int l=strlen(s);
    int sum=0;
    for(int i=0;i<l;i++)
        sum=(sum*131+s[i])%mod;
    while(book[sum]&&strcmp(str[book[sum]], str[k]))
        sum++; //一直向下一个空位找
    if(!book[sum])
    {
        ans++;
        book[sum]=k; //标记该位置是哪一个字符串
    }
}
```

注意下mod的值尽量大一点，这样数组中间的空位就会比较多，这样就算冲突也可以马上找到下一个位置，这就是典型的用空间换时间。

字符串匹配

字符串hash可以快速比较多个字符串是否相等，而且还可以用来字符串匹配，即判断一个字符串是否是另一个字符串的子串。

(1) 先得到匹配串的长度l

(2) 在目标串中O(m)的预处理出来所有长度为l的子串的hash值（用于匹配一般选择自然溢出，减少冲突）

$$\text{sum}[i] = (\text{sum}[i-1] - s[i-1] * 131^{(l-1)}) * 131 + s[i+l-1]$$

(3) 再按照前面的做法来匹配。

使用hash做字符串匹配的时间复杂度为 $n \log n$ ，效率稍慢于kmp算法，时间复杂度为 $O(n+m)$ 。

Friend

有三个好朋友喜欢在一起玩游戏, A君写下一个字符串S, B君将其复制一遍得到T, C君在T的任意位置(包括首尾)插入一个字符得到U. 现在你得到了U, 请你找出S. 如果串S不唯一, 输出NOT UNIQUE

输入:

7

ABXCABC

输出:

ABC

输入:

9

ABABABABA

输出:

NOT UNIQUE

$N \leq 2000000$

Friend

这道题看上去很简单，判断是否由一个字符串*2，中间再插入一个字符组成的，就纯暴力就可以做。插入的位置有五种情况，最前面，前一个字符串中间，两个字符串之间，后一个字符串中，最后面。

但是这道题关键是还需要判断情况是否唯一，比如ABABA，他的原字符串可以是AB，也可以是BA，都符合条件，还有一些更长的，他的情况或者新插入的字符的位置可能有很多种，所以我们需要枚举插入的位置，再判断是否符合条件，为了提交时间效率，我们要用到字符串hash。

Friend

我们先枚举插入字符的位置为 i ，原始字符串长度为 l ，总长度为 n 。有三种情况(五种情况可以简化为三种)。第一种： $i \leq l$ ，第二种 $i = l + 1$ ，第三种 $i > l + 1$ 。

(1) $i \leq l$



T1

T2

T4

我们只需要判断前后两部分的hash值就可以了，因为枚举的时间很大，所以我们计算hash值的时候需要 $O(1)$ 的计算，所以我们先预处理出来整个字符串的hash值，再计算部分的hash值。

$T1 = \text{hash}[i-1];$

$T2 = \text{hash}[l+1] - \text{hash}[i] * \text{pow}[l+1-i];$

$T3$ (前一个字符串真实hash值) $= T1 * \text{pow}[l+1-i] + T2;$

$T4$ (后一个真实hash值) $= \text{hash}[n] - \text{hash}[l+1] * \text{pow}[l];$

Friend

(2) $i == l+1$ 

T1

T2

$T1 = \text{hash}[l];$

$T2 = \text{hash}[n] - \text{hash}[l+1] * \text{pow}[l];$

(3) $i > l+1$ 

T1

T2

T3

$T1 = \text{hash}[1];$

$T2 = \text{hash}[i-1] - \text{hash}[1] * \text{pow}[i-1-1];$

$T3 = \text{hash}[n] - \text{hash}[i] * \text{pow}[n-i];$

$T4 = T2 * \text{pow}[n-i] + T3;$



Friend

这样做看上去是没有问题了，但是在判断重复的时候如果我们只判断前面是否存在前后两个字符串的hash值相等是不准确的，比如AAAAA，无论每次哪一个位置都可以成立，但是实际上原字符串都是AA，所以我们还需要判断一下每次成功的字符串是否是同一个，可以用hash，也可以用hash，如果用hash就必须取余，用map比较慢



B e a d s

有一个长度为 n 的串，每个位置分别有一个数字，按照每段长度为 k 来切(最后如果剩余小于 k 的长度就丢掉了)，问如何选择 k 的值才能保证不同的段数最多，注意串是可以反转的，反转之后相同的串也是相同的串，比如 $(1, 2, 3)$ 和 $(3, 2, 1)$ 。

比如1 2 3 3 2 1

按照 $k=1$ 来切，最后不同串为 (1) , (2) , (3)

$k=2$, 不同串为 $(1, 2)$, $(3, 3)$

$k=3$, 不同串为 $(1, 2, 3)$

$k=4$, 不同串串为 $(1, 2, 3, 3)$

。 。 。

$n \leq 200000$

B e a d s

该题需要判断每个分割出来的字符串是否相等，比较简单，先预处理出来之后，用字符串hash就可以搞定，但是该题要求不考虑正反，所以还要反着hash一遍，其实没有必要比较两次，直接用正hash*反hash值来存储就可以了，因为后面需要用到前面的hash值，所以要判重，还是考虑map或者hash。注意分割的时候一定要优化，否则会超时，该题不能分解质因数，但是可以考虑当前最大不重复子串数 $\leq n/k$ ，一样可以优化

通过前面的学习，我们会发现hash主要用来快速判断字符串是否相等，大部分题只要涉及到子串相等，字符串相等，都可以用字符串hash来做。

但是一旦设计字符串排序，那么hash肯定是做不了的！！！！



Hash 练习题

洛谷 3370 字符串hash
MSOJ 1465 图书管理
BZOJ 3916 friends
BZOJ 2795 A Horrible Poem
BZOJ 2081 Beads
BZOJ 2084 Antisymmetry
LOJ 10041 门票
LOJ 10042 收集雪花



KMP算法

KMP算法是字符串单串匹配中效率最高，代码最简洁，最容易记忆的算法，但是也是最容易忘记，考试时最容易出问题的算法。

在KMP算法中，最重要的一个东西叫next指针(最长公共前后缀长度)。比如ababa, 他的前缀有a, ab, aba, abab, 后缀有a, ba, aba, baba, 那么他们的最长公共前后缀长度为3. 但是如果我们需要一次求出字符串中前i个字符的next, 我们应该如何快速求呢? 也就是我们需要找到 $\text{next}[i]$ 和 $\text{next}[i-1]$ 的关系。

(1) 如果 $\text{next}[i-1]=j$, 那说明 $s[j-1]=s[i-1]$, 那么如果后面也相等, 即 $s[j]=s[i]$, 则最长公共前后缀 $\text{next}[i]=\text{next}[i-1]+1$.

KMP算法

(2) 但是如果 $s[i] \neq s[j]$ 呢?



那我们就需要在 $s[0]—s[j-1]$ 中继续去找, 如果我们单独考虑字符串 $s[0]—s[j-1]$, 我们就会发现, 我们可以直接跳到 $s[j-1]$ 处的next指针的位置, 因为只有这里才有可能满足公共前后缀相等(图中黄色部分), 再比较开始和末尾黄色部分的下一个字符是否相等, 如果相等, next值+1, 否则继续跳到黄色字符串的next指针的位置, 直到不存在为止。

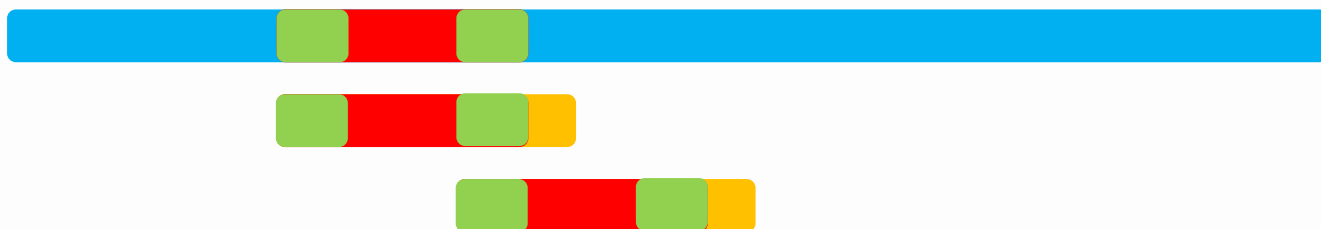
KMP算法

```
void nexts()  
{  
    int j;  
    j=nextn[0]=0;  
    for(int i=1;i<n;i++)  
    {  
        while(j&& s[j]!=s[i])  
        {  
            j=nextn[j-1];  
        }  
        if(s[i]==s[j])  
            j++;  
        nextn[i]=j;  
    }  
}
```

注意：next是
关键字，所以不
能设为全局变量。

KMP算法

关键是最长公共前后缀有什么用呢？
我们来模拟一下匹配过程。



当继续匹配失败的时候(黄色和蓝色部分)，常规做法就是把下面这个字符串向后面挪动一位，重新匹配。但是其实我们可以直接挪到下一个位置，即直接比较已经匹配的最后位置的next指针的位置。

证明很好证明，反证，如果前面还存在相等的位置。。。。

KMP算法

```
int j=0;
for(int i=0;i<l1;i++)
{
    while(s1[i]!=s2[j]&&j)
    {
        j=nexts[j-1];
    }
    if(s1[i]==s2[j])
    {
        j++;
        if(j==l2)
        printf("%d\n",i-l2+2);
    }
}
```

j 表示当前需要匹配的位置，如果不可以匹配，应该去找前一个可以匹配的next指针($j-1$), $next[j-1]$ 表示最后一个可以匹配的位置，那么应该继续比较下一个位置为 $next[j-1]-1+1$.

剪花布条

一块花布条，里面有一些图案，另有一块直接可用的小饰条，里面也有一些图案。对于给定的花布条和小饰条，请计算，能从花布条中尽可能多的剪出几块小饰条

输入：

abcde a3

aaaaaa aa

#

输出：

0

3

剪花布条

其实这道题就是一道kmp的裸题，每次用kmp匹配之后，当把该子串截取出来之后，为了避免重叠，只需要把j(当前匹配的长度)置为0就可以了，这样就不会重叠并且满足要求。



Power string

给定若干个长度小于等于1000000000的字符串，询问每个字符串最多由多少个相同的子串重复连接而成。入ababab最多由3个ab连接而成。

输入：

abcd

aaaa

ababab

...

输出：

1

4

3



KMP求最小循环节

很容易想到第一种思路是先按照位数分解质因数，然后再用hash值来判断是否是循环节，但是这样做效率不是很高，涉及到质因数分解以及快速幂，总的时间复杂度会多一个log，而且hash验证的时候比较麻烦，所以我们考虑另一种解法。

在学习KMP的时候，我们理解了next数组，也就是公共前后缀。我们来看几个有循环节的字符串：

ababab:next[6]=4 循环节长度为2

abcabc:next[6]=3 循环节长度为3

ababaab:next[7]=2 无循环节

aaaaaa:next[6]=5 循环节长度为1

好像有点规律，我们再画图看一下。

KMP求最小循环节

字符串长度为a，最后一个字符的next数组的值为b。
根据图像很容易看出来如果两端覆盖没有交叉，肯定无循环节。

我们假设一个字符串有循环节，循环节长度为b，
那么很明显，最后一个字符串的next数组的长度为a-b。

反过来，我们知道字符串字符串的长度和next数组，是可以直接求出循环节的，但是，我们要如何判断是否有循环节呢？很明显，既然是循环节，那么肯定要整除。所以我们只需要判断

$$1 \% (1 - \text{next}[1]) == 0 \text{ 就可以了。}$$

BZOJ 1355

给定一个字符串的子串，已知这个原字符串是由某个字符串不断自我连接形成的，问该字符串的最短长度是多少。

输入：

8

cabcabca

输出：

3

他可以由cab或者bca连接而成，比如三个cab连接而成cabcabcab，给定的字符串cabcabca是他的子串。

BZOJ 1355

其实该题还是用kmp求循环节，最短循环节的长度仍然是 $l - \text{next}[l-1]$ ，画一个图就出来了，是他的子串不影响原来的求法。



$\text{next}[\text{len}]$

一个完整的最短循环节

如果是前面残缺了一部分和后面残缺一部分是相同的。

BZOJ 1511

先定义字符串的周期：对于两个字符A,B，如果A是B的前缀，并且B是AA的前缀，那么A就是B的周期，如果不存在，周期就是0，比如abab和ababab都是abababa的周期，ababaaba的最大周期是ababab，先给出一个串，求出他所有前缀的最大周期之和。

输入：

8

babababa

输出：

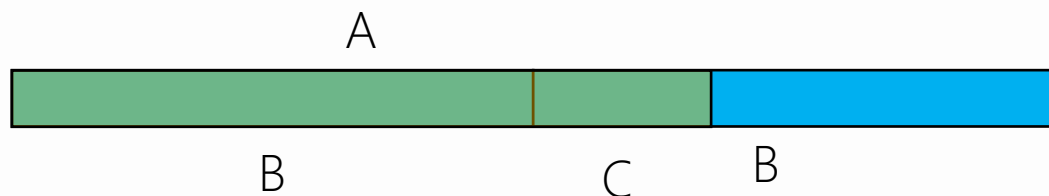
24

$$0+0+2+2+4+4+6+6=24$$

$$n \leq 1000000$$

BZOJ 1511

一般情况下，只要涉及到前后缀的问题大部分都可以用kmp来求解，对于可以用kmp求解的问题，如果以前没有遇到过或者不是很能理解，那么最好的办法就是画图。



画图之后我们很容易就看出来对于A串来说，只需要A有公共前后缀就可以了(C)，这就满足了B是A的前缀，同时A是BB的前缀，该题要求最大周期，也就是最大的B，很明显要C最短，实际上就是已知求该位置的next数组，直到最后一个不为零的位置为止。但是直接求最坏时间复杂度为 $n*n$ ，所以我们需要继承上一个答案的最短公共前后缀(要么+1，要么继续向前找)。

动物园

求num数组：对于字符串S的前i个字符构成的子串，既是他的后缀同时又是他的前缀，并且还后缀和前缀不重叠，讲这种字符串的数量记做num[i]，例如S=aaaaa，则num[4]=2（a和aa），aaa虽然满足前后缀相同，但是不满足不重叠，所以不算。先要求一个字符串中num[i]+1之积。

输入：

3

aaaaa

ab

abcababc

输出：

36

1

32

动物园

该题题目中说的很明显，肯定是用kmp求解，很容易想到暴力方法，对于每个位置求next数组的时候，再一直往next的位置跳，判断 $\text{next} \times 2$ 和当前长度的大小，如果满足就条件，就累加，这样就可以直接求出来每个位置num的值，时间复杂度为 $n \times n$ 。很明显过不了。

看数据范围1000000，那么很明显就是 $O(n)$ 的算法，也就是类似于递推，考虑 $\text{num}[i]$ 和 $\text{num}[i-1]$ 的关系或者 $\text{num}[i]$ 和 $\text{num}[\text{next}[i]]$ 的关系。

在寻找关系的时候我们会发现，如果要考虑是否重叠，关系很复杂，一会+1一会不加，单如果先不考虑重叠，让 $\text{num}[i]$ 直接表示公共前后缀的数量，那么关系就很清晰明了：

$$\text{num}[i] = \text{num}[\text{next}[i] - 1] + 1$$

为了方便，这里的前后缀都包含本身，所以 $\text{num}[0] = 1$

动物园

接下来就是要去除掉重叠部分了，对于位置 i ， $\text{next}[i]=j$ ，如果 $j*2 > i+1$ ，那么说明当前的 j 是不满足条件的，所以我们直接让 $j=\text{next}[j-1]$ 。如果满足条件，我们以前求的 $\text{num}[j]$ 的值就是我们需要的答案(包含自身的公共前后缀的数量)，有人可能会觉得每次查找的时候 $j=\text{next}[j-1]$ ，万一数据很坑呢？实际上不影响，对于每次 i 来说， $j \leq i/2$ ，那么对于下一个位置 $i+1$ ， j 的变化只有两种情况 $j+1$ 或者 $j=\text{next}[j-1]$ ，前面 j 最多增加一位，那么就算 $j > i/2$ ，那么最多一次 $j=\text{next}[j-1]$ 就可以保证 $j \leq i/2$ ，所以 `while` 完全可以写成 `if`，不影响结果。

动物园

```
for(int i=1;i<l;i++)
{
    while(s[i]!=s[j]&& j)//对于上一次的最大不重复匹配长度为j
    //下一个的最大匹配只有两种情况，第一种是j++，不可能更大了
    //还有一种是s[i]!=s[j]，说明还需要到前面去找，怎么找到，用nextn数组。
    {
        j=nextn[j-1]; //在前面找最长公共前后缀，原理和next数组一样
    }
    if(s[i]==s[j])
        j++;
    while(2*j>i+1)//判断当前匹配长度是否过了一半，这里的while实际上在后面的操作中就是if的作用
    //因为我每次都保证了j<=一半，下一次最多j++超过一半，只需要一次next就小于一半，所以不会超时
    {
        j=nextn[j-1];
    }
    if(j!=0)
        ans=ans*(num[j-1]+1)%1000000007;
}
```


KMP 练习题

洛谷 3375 KMP匹配

Poj 2406 Power Strings

Poj 2752 Seek the Name

Hdu 剪花布条

BZOJ 1355 Radio Transmission

BZOJ 1511 Words

BZOJ 3620 似乎在梦中见过的样子

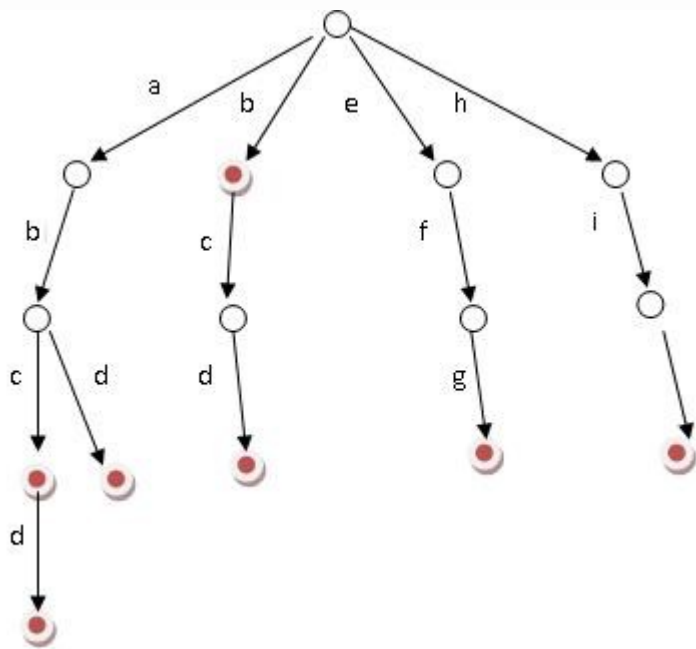
BZOJ 3942 Censoring

洛谷 2375 动物园

KMP算法主要用于求包含字符串匹配以及公共前后缀的问题。

字典树

如果存在很多字符串，为了存储方便和查询省时间，我们可以把这些字符串构造成一颗字典树，即多个单词相同前缀的部分用同一条路径表示。



单词abcd和abd有相同的前缀ab，所以前缀在一条路径上，后面不一样的部分分叉，因为每个位置最多26个字母，所以空间复杂度为 $O(m) * 26$ ，远小于 $O(m * n)$ 。

字典树

```
void build(int id,char *s)//建树
{
    int l=strlen(s);
    for(int i=0;i<l;i++)
    {
        int j=s[i]-'a';
        if(tree[id].next[j]==0)
        {
            k++;
            tree[id].next[j]=k;
        }
        id=tree[id].next[j];
    }
    tree[id].flag=1;//表示该位置是一个完整的单词
    return ;
}
```

字典树的应用

一：求前缀：

- (1) 给定 m 个单词和 n 个前缀，分别求有多少个单词包含这些前缀
- (2) 给定 m 个字符串，判断是否存在一个单词是另一个单词的前缀

二：判断单词是否出现过

- (1) 给定一篇文章，再给定一些单词，判断这些单词是否在文章中出现过
- (2) 给定一些单词，再给定一篇文章，问这些单词是否可以完全构成这篇文章

三：树上异或值



Phone List

给定n个长度不超过10的数字串，问其中是否存在两个数字串S, T，使得S是T的前缀。 $N \leq 10^5$

输入：

2

3

911

97625999

91125426

5

113

12340

123440

12345

98346

输出：

YES

NO



Phone List

该题因为是求前缀，所以KMP没有任何意思，还不如暴力比较，如果用字符串hash，因为hash主要是判断两个字符串是否相等，如果要判断一个字符串和另一个字符串的前缀是否相等，还是只能一个一个比较，时间复杂度都是 $O(n^2)$ 。

所以我们考虑字典树，思路很简单，边建树边判断，对于当前新加入的字符串，在插入字典树的过程中，如果遇到了flag标记，就表示前面出现了单词是当前单词的前缀，那么就存在，如果所有单词插入完之后都不存在，那么就不存在。

最大异或值

给定N个数，从中选择两个数进行异或，问最大异或值是多少？

输入：

5

2 9 5 7 0

输出：

14

$N \leq 100000$, 值 $< 2^{31}$

最大异或值

首先，题目一看完暴力方法就想出来了，枚举异或呗，但是要AC呢？很容易想到一种思路，为了让异或的值最大，那么很明显要让最高位最大，对于异或来说，只有0和1两种，那么很明显就是如果原来这个数高位是1，那么我们就尽量先选择该位为0的，如果是0，我们就选则是1的，当然，如果不存在满足要求的那就没得选。为了优化，我们一般采用二分，但是我们又没有办法按每一位二分。

我们在模拟过程中会发现，比如高位需要一个1来异或，那么可以选的数的范围就减小了很多，那么下一位需要一个0，那么符合条件的数又少了了很多，这样一直一下，一定只有一个数满足条件。

关键是怎么来实现呢？怎样才能把前缀每一位的值相同的数放在一起呢？

01trie

其实可以用字典树来实现，当然这里是01trie，即树上的每一个结点表示数的每一位，这样我们就可以把前缀值相同的数放在一起了。比如101，要让异或值最大，我们首先最好给他匹配一个0，我们就在字典树中第一层(0结点下的那一层)查找是否存在有0的结点，如果存在，那么第一位找到了，如果不存在，那就只能找1了。接下来，我们需要找1，和前面一样的办法。每次找到答案之后把这条路径上找到的值按位累加上来到最后就是真实值。

思路很简单，代码也很简单。

01trie

```
struct node
{
    int son[2];
}tree[maxn<<5];// 每一个数可以拆分成31位，所以要*32
void build(int id,int x)
{
    for(int i=30;i>=0;i--)
    {
        int t=((x>>i)&1);// 数x的第i位的值
        if(tree[id].son[t]==0)// 如果不存在结点，新建一个
        {
            k++;
            tree[id].son[t]=k;
        }
        id=tree[id].son[t];
    }
}
```

01trie

```
int find(int id,int x)
{
    int ans=0;
    for(int i=30;i>=0;i--)
    {
        int t=!((x>>i)&1);//如果该位是1，那么再来一个数是0最好，如果是0，再来一个1最好
        if(tree[id].son[t])//如果有最优选项
        {
            id=tree[id].son[t];
            ans+=t<<i;//累加该位的值
        }
        else if(tree[id].son[t^1])//如果没有
        {
            id=tree[id].son[t^1];//如果没有也只能将就了
            ans+=(t^1)<<i;
        }
    }
    return ans;//把找到的答案返回回去
}
```

REBXOR

给定一个含 N 个元素的数组 A ，下标从 1 开始。请找出下面式子的最大值：

$$(A[l_1] \oplus A[l_1 + 1] \oplus \cdots \oplus A[r_1]) + (A[l_2] \oplus A[l_2 + 1] \oplus \cdots \oplus A[r_2])$$

其中 $1 \leq l_1 \leq r_1 < l_2 \leq r_2 \leq N$ 。式中 $x \oplus y$ 表示 x 和 y 的按位异或

输入：

5

1 2 3 1 2

输出：

6

REBXOR

遇到这种包含数学等式的题一定要先化简，该等式可以理解为找到两个区间(不重叠)，让这两个区间的异或之和的数学和最大。

这里需要的是区间异或和，而字典树是前缀，所以我们先把区间转换成前缀，然后发现。。。没区别， $[l, r]$ 的异或和等价于 $[1, l-1]$ 的异或和，再异或上 $[1, r]$ 的异或和，因为前面的 $[1, l-1]$ 出现过两次，异或=0。

第一个问题解决了，但是还有一个问题，不是求一个区间最大，而是两个区间异或和之和最大，所以我们还不能先选一个最大的，再重剩下的里面再选一个最大的。

那就按照动态规划的思想，先枚举一个断点 i ，左边区间端点 $\leq i$ ，右边端点 $> i$ ，

REBXOR

所以思路就很简单了，以 i 为断点，左边只能选择 $[1, i]$ 组成的前缀，右边选择的是 $[i+1, n]$ ，比较麻烦，所以我们完全可以倒着再建一个树，这样也变成了前缀，当第 j 个单词添加进来的时候，我们先求一个最大异或，在把该单词添加进来，和前面的最大异或值去一个 $\max$ 当做 $I[j]$ 。倒着也是一样，这样就保证了我们取数的时候分别为了 i 的两边，没有越界。

L 语言

题目大意：给你n个单词，m篇文章，问该文章是否能被理解(如果该文章能被其中的某些单词完整的拼凑起来那么就认为可以被理解)。如果不能被完全理解，那么输出最多可以被理解的前缀

输入：

4 2

abc

abcd

def

efg

abcdef

abcdefg

输出：

6

7

L 语言

该题想到思路很简单，先把所有单词建一个字典树，然后再用文章去匹配，看一下最多可以匹配到哪个位置，关键是该题有一个细节很简单？假设字典树的一条路径上有多个单词，比如a, ab, abc, abcd，那么我该如何匹配？

有人可能觉得，那还不简单，能继续向下匹配肯定继续匹配噻？很明显样例就过不了。那我匹配到都是一个完整单词我就不匹配了，样例还是过不了，由于后面的不确定性，所以无论每次主观的判断匹配还是不匹配答案都是错误的，那应该怎么办？

很简单，都去尝试一遍。

L 语言

也就是说，先匹配一条链，把这条链上可能是单词结尾的地方都做一个标记。

11 12 13

然后把这些被标记过的点都当做起点继续匹配，把找到的一条链匹配完，继续标记可能出现的所有结尾，一直这样做下去，每次以当前被标记的点为起点，最终找到的可以匹配的最长的一定是答案。

L 语言

也就是说，先匹配一条链，把这条链上可能是单词结尾的地方都做一个标记。

11 12 13

然后把这些被标记过的点都当做起点继续匹配，把找到的一条链匹配完，继续标记可能出现的所有结尾，一直这样做下去，每次以当前被标记的点为起点，最终找到的可以匹配的最长的一定是答案。

字典树练习题

洛谷 2580 于是他错误的点名开始了

洛谷 2292 L语言

洛谷 2922 秘密消息

洛谷 3061 疯狂的栅栏

洛谷 3879 阅读理解

Poj 3630 Phone List

LOJ 10050 The XOR Largest Pair

BZOJ 4260 REBXOR

洛谷 4551 最长异或路径

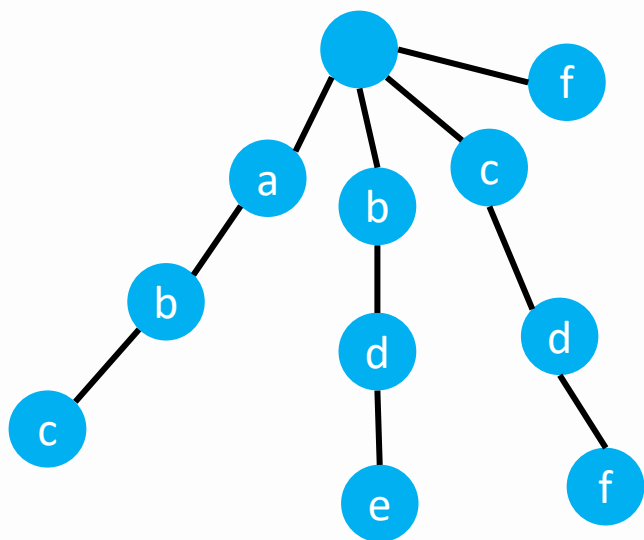
洛谷 3065 First

洛谷 4735 最大异或和

洛谷 3449 PAL-palindromes

AC自动机

字典树的应用大多体现在求前缀上，如果是求多串匹配的子串，就需要用AC自动机来实现，AC自动机本质就是KMP+字典树。



我们先把子串建字典树，假设主串为abde，那么先匹配了ab，发现d不可以匹配，和kmp算法一样，我们没必要挪一位从b开始匹配，而是是否存在以当前串ab的后缀为前缀的子串，这样的子串前面肯定是满足要求的，接着匹配，这里有b，那么接着匹配就可以了。

A C 自动机

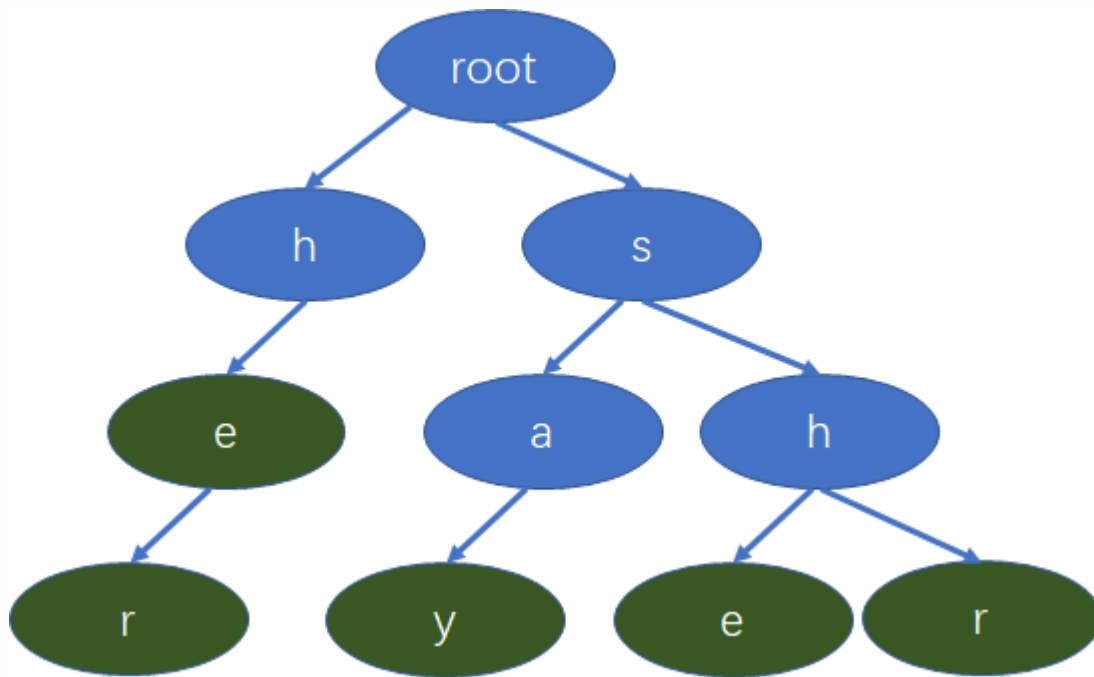
关键是失配指针如何建立呢？首先，第一层的结点的失配指针肯定是根结点(0结点)，只有一个字符不存在公共前后缀，对于后面层次的点，如果可以匹配，那么继续匹配，并且判断他的上一个点的失配指针的下一个结点是否存在和他相同的点，如果存在，就指过去，否则指向0结点。如果不可以匹配，那么和next数组一样，跳到他的公共前后缀(失配指针指向的位置的下一个位置)继续判断，直到可以匹配或者失配指针指向0结点为止。

注意，失配指针的建立是按字典树的层来依次建立的，所以要用一个队列来分别存储每一层的结点，并且给他们建立失配指针。

算法流程

第一步：建立字典树

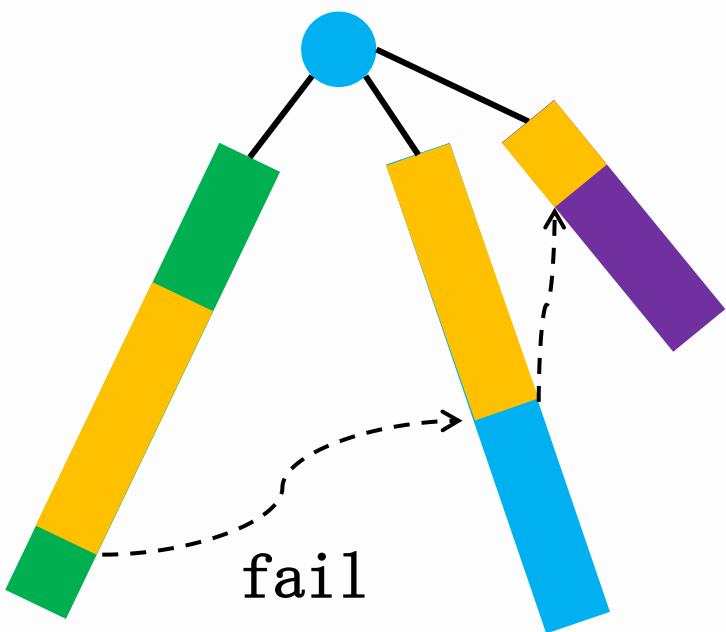
输入: she, he, say, her, shr



算法流程

第二步：对每个结点寻找失配指针

AC自动机的失配指针相当于kmp的next数组。也就是对当前字符串中的后缀找一个公共前缀，这样的话，如果该字符串匹配到当前位置，就可以直接跳到他的失配指针继续匹配。



失配指针的作用在于，当匹配到左边字符串的黄色部分的时候，如果发现不可以继续匹配，只是代表该链的字符串不是给定字符串的子串，那么就需要继续匹配，和kmp一样，没必要换到下一个字符串开头继续比较，因为通过失配指针，我们可以发现右边字符串的黄色部分已经被匹配过了，可以接着匹配。

算法流程

那失配指针应该如何建立呢？

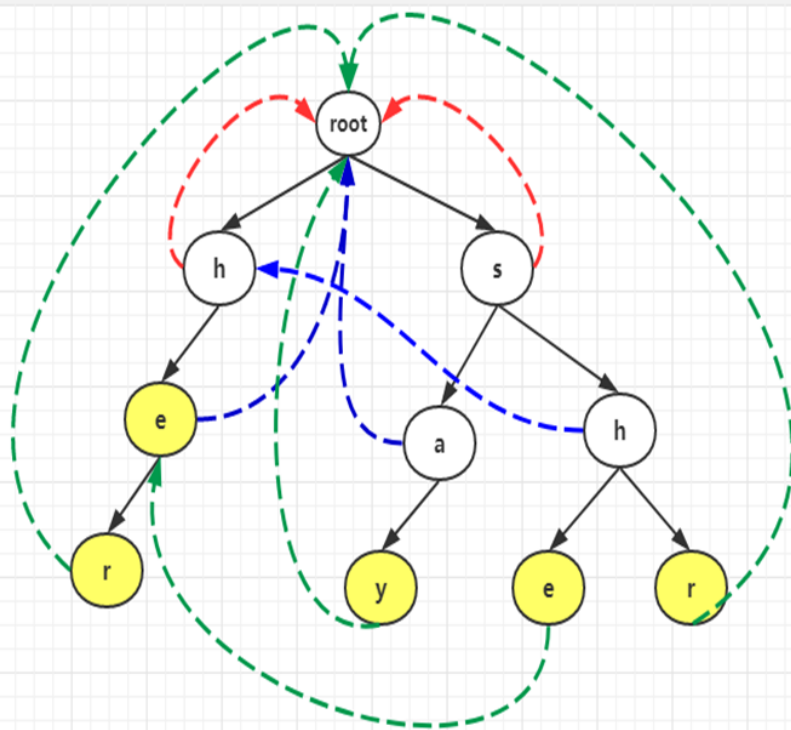
首先，对于第一层的点，0号节点下方的结点，因为只有一个字符，所以不存在公共前后缀，那么很明显他们的fail指针指向0号结点

那么下方的结点，他们的fail指针指向哪呢？

我们先找到他的父亲i结点的fail指针k，那么此时以i为后缀的字符串和以fail结尾的字符串时相同的，那么我们就只需要比较k结点下方是否存在一个与该结点相同的结点就可以了。如果存在就指过去，如果不存在呢？直接指向0号结点吗？想一下kmp？是不是应该继续找公共前后缀的内部的公共前后缀，在ac自动机中就体现在继续找fail指针，如果找完都没有找到匹配的，那么就指向0号结点。

算法流程

she
he
say
shr
her
yasherhs



算法流程

如图所示，首先root最初会进队，然后root出队，我们把root的孩子的失败指针都指向root。因此图中h,s的失败指针都指向root,如红色线条所示，同时h,s进队。

接下来该h出队，我们就找h的孩子的fail指针，首先我们发现h这个节点其fail指针指向root,而root又没有字符为e的孩子，则e的fail指针是空的，如果为空，则也要指向root,如图中蓝色线所示。并且e进队，此时s要出队，我们再找s的孩子a,h的fail指针，我们发现s的fail指针指向root,而root没有字符为a的孩子，故a的fail指针指向root,a入队，然后找h的fail指针，同样的先看s的fail指针是root,发现root又字符为h的孩子，所以h的fail指针就指向了第二层的h节点。e,a,h的fail指针的指向如图蓝色线所示。

此时队列中有e,a,h,e先出队，找e的孩子r的失败指针，我们先看e的失败指针，发现找到了root,root没有字符为r的孩子，则r的失败指针指向了root,并且r进队，然后a出队，我们也是先看a的失败指针，发现是root,则y的fail指针就会指向root.并且y进队。然后h出队，考虑h的孩子e,则我们看h的失败指针，指向第二层的h节点，看这个节点发现有字符值为e的节点，最后一行的节点e的失败指针就指向第三层的e。最后找r的指针，同样看第二层的h节点，其孩子节点不含有字符r,则会继续往前找h的失败指针找到了根，根下面的孩子节点也不存在有字符r,则最后r就指向根节点，最后一行节点的fail指针如绿色虚线所示。

算法流程

第三步：文本串的匹配

当AC自动机搭建好了之后，接下来就是匹配了，对于一个给定字符串，比如要求多少个文本串在其中出现过，那我们就可以先匹配，因为比如有abcd, bcd, cd, 当abcd匹配完之后实际上bcd, cd都匹配完了，所以在匹配过程中，找到一个字符的时候，应该继续去找他的失配指针以及失配指针的失配指针，判断是否是单词的结尾，当找完之后，再回到最开始的那条链继续往下面找。

因为在我们搭建AC自动机的时候，因为把每个字符串的最后一个字符的下一个位置链接到其失配指针的下一个位置了，所以只需要一直遍历next就可以了。

AC自动机

```
void fail_point(int id){
    while(!q.empty()) q.pop();
    for(int i=0;i<26;i++){
        int j=tree[id].next[i];
        if(j!=0)//把第一层结点放入队列中，即源点直接连接的点
        {
            tree[j].fail=id;//第一层指针的失配指针都是根结点
            //因为第一层结点肯定都是各不相同的，所以第一层结点只能指向他本身。
            q.push(j);//我们依次把一层一层的结点都放进去。
        }
    }
    while(!q.empty()){
        int now=q.front(); q.pop();
        for(int i=0;i<26;i++){
            int j=tree[now].next[i];
            if(j==0)//now 下面的当前位置没有结点了
            {
                tree[now].next[i]=tree[tree[now].fail].next[i];
                continue;
                //如果当前结点下方没有结点了，那么为了避免重复，我们应该找到他的子串的接下来的位置继续匹配
                //比如 abcd bcde cdef
                //当abcd匹配完之后，实际上bcd也匹配完了。我们应该接着bcd之后去匹配e。
                //同样的，当e匹配完之后，我们应该接着cde之后匹配f
                //我们会发现前一个串的后缀等于后一个串的前缀的时候我们才可以匹配，否则不可以
            }
            tree[j].fail=tree[tree[now].fail].next[i]; q.push(j);
        }
    }
}
```

A C 自动机

```
void search(int id ,char *s)
{
    int l=strlen(s);
    for(int i=0;i<l;i++)
    {
        int j=tree[id].next[s[i]-'a'];
        while(j!=0&&tree[j].flag!=-1)
            //flag即表示是单词，也表示了单词的个数
            //如果该单词匹配过了，再下次仍然可以匹配的时候就不在匹配了
            //否则一个单词会重复计算。
            //比如ab ababab,ab只应该算一次
        {
            ans+=tree[j].flag;//对于非单词结尾，都是0，对结果不影响
            tree[j].flag=-1;//算过了就不再算了，标记为-1.
            j=tree[j].fail;//找到还有没有这个字符
            //比如当前是***a,他会继续查找其他链里面是否还有前缀a,
            //相当于一层一层的找
        }
        id=tree[id].next[s[i]-'a'];//相当于第一条链为主链，依次查找是否有相同
        //不能写成j，因为j改变了
    }
}
```

单词

给定一篇文章的所有组成单词，问文章中的每一个单词在文章中出现了多少次。

输入：

3

a

aa

aaa

输出：

6

3

1

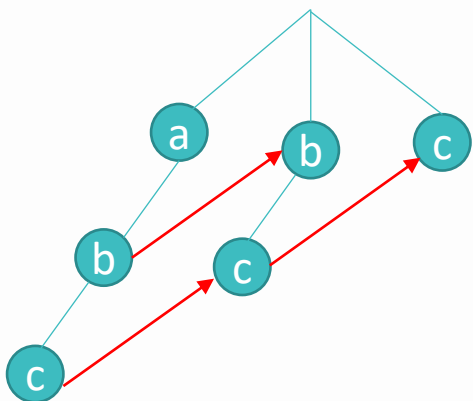
原文为 a aa aaa, 那么单词a出现了6次，单词aa出现了三次(在aaa中算出现了两次)，aaa出现了一次。

单词

该题是多串匹配中的子串匹配，那么一般情况下就是AC自动机来求解，字典树是肯定不行的，字典树只能做前缀那种。

关键是该题统计答案的时候，比如问aa在aaaa中出现了多少次，并不是2次，而是3次。

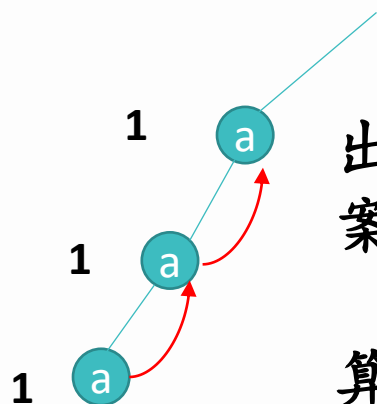
比如下面这种情况，问c出现了多少次，很明显是3次，前面我们已经说了fail指向的位置表示到该位置的前缀=当前位置的后缀，那么很明显是包含关系。



我们会发现，如果把fail指针反过来，那么这条路径上的cnt的数量之和就刚好等于该单词出现的次数。
即 $\text{sum}[\text{fail}[i]] += \text{sum}[i]$

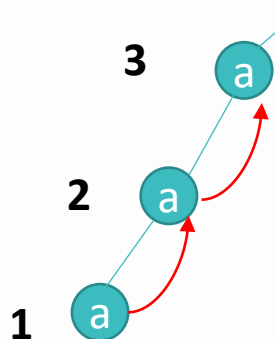
单词

但是我们换成题目中的样例就会发现问题。



按照前面的算法，单词a出现的次数是3次，aa出现的次数是2次，aaa出现的次数是1次。明显答案是错误的。

因为这几个单词是包含关系，而我们却没有计算进去，我们实际上计算的时候只考虑了a在每个单词中是否以后缀的形式出现过，而没有考虑其他情况。



所以我们应该在建树的时候，每插入一个新的单词，这条路径上的cnt++，表示新单词经过的路径都是被他包含的。

最短母串

给定n个字符串，现要求你找出一个最短的字符串S，满足这个n个字符串都是字符串S的子串。

输入：

2

ABCD

BCDABC

输出：

ABCDABC

$N \leq 12$, 长度 ≤ 50 .

最短母串

该题一定要注意，数据特别小， $n \leq 12$ ，长度 ≤ 50 ，听说该题可以用状压dp做出来(不会)，但是实际上建一个AC自动机之后暴力枚举就可以A了。

我们需要答案是**最短的+字典序最小的**。

既然要字典序最小的，那我就从A—Z枚举，要求最短的，最短，那考虑BFS拓展，所以答案就有了，最后还需要判断是否包含所有单词，也简单，用过二进制状态标记就可以了。

所以思路就是从A—Z枚举每一位，然后用当前字符串状态去判断有多少被匹配过了，然后把当前的每一层都放入队列中去拓展，直到第一次拓展到所有字符串都被标记位置。

时间复杂度为 $< (12 * 50 * 26) * (26 * 50) = 2000w$



AC自动机练习题

洛谷 3808 AC自动机
洛谷 3796 AC自动机
洛谷 5357 AC自动机
BZOJ 3172 单词
HDU 2222 Keywords
BZOJ 4327 玄武密码
BZOJ 3940 Censoring
BZOJ 1195 最短母串
BZOJ 2938 病毒
BZOJ 1030 文本生成器
洛谷 2336 喵星球上的点名



二分 + hash

根据前面的hash的学习，感觉hash作用也不是很大，要么效率不高，要么准确度不高，并且有很多可以替代的方法，接下来我们来学习一种用hash骗分的方法——二分+hash。

求解一个字符串中的最长回文子串。

输入：

abcbabcbabcba

abacacbaaab

输出：

13

6

最长回文子串

比如dcbaaabcf

当我们枚举到第二个a的时候，先枚举长度4
发现dcba和abcf不相等，长度减半为2

发现ba和ab相等，但是此时并不是就完了，
此时我们只是找到了回文串，但是不一定是最长回文串，所以此时回文长度在 $[2, 4)$ 之间，我们再枚举长度3，发现，仍然匹配，此时区间长度为1，就没有必要再枚举了，所以总长度为 $1+3*2=7$.

二分 + hash

1: 和前面用hash做字符串匹配一样，先预处理前缀hash之和。但是值得注意的是注意方向。

比如：abcdcda

算前缀是 $a \rightarrow c$ ，算后缀应该还是 $a \rightarrow c$ ，所以应该分别预处理正向和反向的前缀hash值。

```
zheng[0]=0 ; fan[len+1]=0;
for(int i=1;i<=len;i++)
zheng[i]=zheng[i-1]*131+(str[i]-'a');
for(int i=len;i>=1;i--)
fan[i]=fan[i+1]*131+(str[i]-'a');
```

二分 + hash

2: 枚举可能是回文中心的字符，注意可能是该字符，也可能是该字符后面的那个空位，所以两种情况都需要分别枚举。

```
void Single(int i) //以i为对称中心
{
    int l=0, r=min(i-1, len-i); //枚举回文长度
    while(l<=r)
    {
        int mid=(l+r)/2;
        if(check(i-mid, i-1, i+mid, i+1))
            ans=mid, l=mid+1;
        else
            r=mid-1;
    }
    Max=max(Max, ans*2+1);
}
```

二分 + hash

```
void Double(int i) //以i后面的空位置为对称中心
{
    int l=0, r=min(i-1, len-i-1);
    while(l<=r)
    {
        int mid=(l+r)/2;
        if(check(i-mid, i-1, i+mid+1, i+2))
            ans=mid, l=mid+1;
        else
            r=mid-1;
    }
    Max=max(Max, (ans+1)*2);
}
```


最长双回文串

给定一个长度为 n 的字符串 S ，问 S 的最长双回文子串 T ，即将 T 分为两部分 X, Y ， $|X|, |Y| \geq 1$ 且 X, Y 都是回文串。

输入：

abcbabcbabcba

abacacbaaaaab END

输出：

Case 1: 13

Case 2: 6

字符串长度 ≤ 100000

最长双回文串

该题既然是要求回文子串，那么我们考虑二分+hash，关键是并不是说这个字符串就是回文串，而是由两个回文串组成的，所以直接二分+hash明显不得行。

那我们只能分别枚举这两个回文串，判断这两个回文串是否可以组成一个子串，我们来画图演示一下：

一：两个回文串不相交

无法构成一个字符串，不考虑

二：两个回文串刚好相切

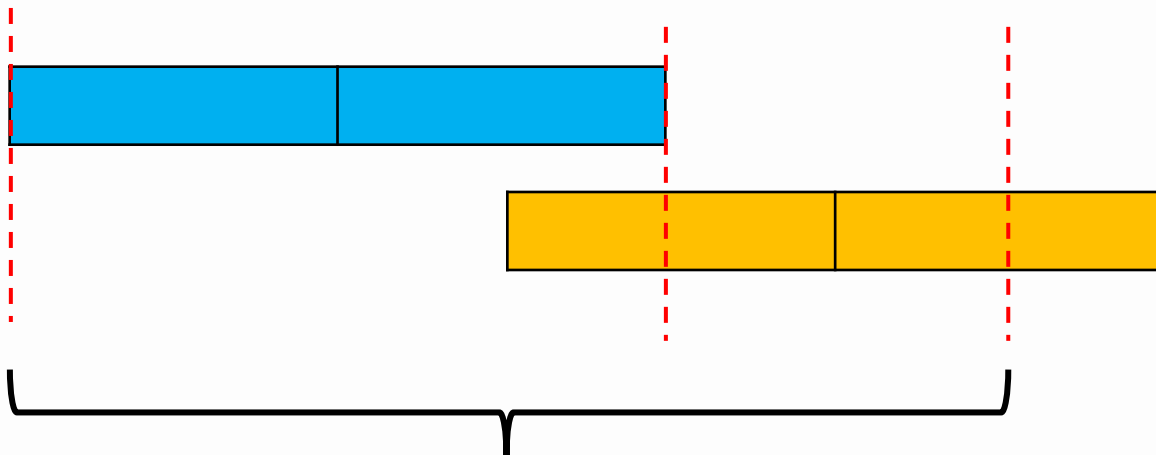
那么构成双回文串就是两个回文串的长度之和

最长双回文串

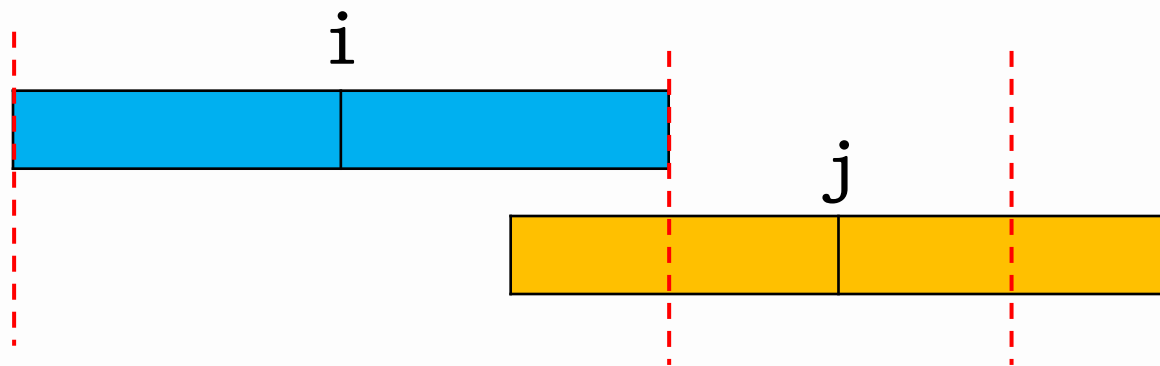
三：如果两个回文串相交



这两个回文串是完全可以组成一个双回文串的，如下图



最长双回文串



蓝色长度为 $l1$
黄色长度为 $l2$

如果蓝色部分取整个回文串，那么我们只要求出黄色部分的回文串长度就可以了， $(j-i-l1)*2$ ，那么总的双回文串长度为 $(j-i-l1)*2+l1*2= (j-i)*2$ 。

我们会发现和回文串的长度没有任何关系，但是我们首先得确定两个回文串相交，所以我们先用二分+hash预处理出来所有位置的最长回文串的长度

$f[i]$:表示以 i 为回文中心的一半最长回文串长度

最长双回文串

但是仅仅有这个是不够的，我们还需要知道如果要覆盖某点 i ，左边最远的那个点是哪一个点，右边最远的哪一个点是那一个点，这样我们就能得到最远的两个点 i, j ，就可以知道答案了。

首先，我们先求出如果要覆盖到点 i ，最左边的回文中心是： $l[i+f[i]-1]=\min(l[i+f[i]-1], i)$ ；

同样的，如果要求最右边的回文中心，可以倒着从 n 往 1 枚举。

$r[i-f[i]+1]=\max(r[i-f[i]+1], i)$ ；

因为每个回文串的长度都不能为空，所以一定要判断一下是否会出现一个回文串刚好覆盖到另一个回文串中心的这种情况。

该题为了方便，在hash之前在每个位置添加了一个#，这样回文串是一定有中心的，并且一定是奇数数。就不需要分情况讨论了。



练习题

P0J	3974	Palindrome
洛谷	4555	最长双回文串
BZOJ	3796	追妹子
BZOJ	2258	文本校对
BZOJ	3790	神奇的项链
BZOJ	2084	Antisymmetry
BZOJ	3555	企鹅QQ





练习题

洛谷	1381	单词背诵
洛谷	2543	奇怪的字符串
洛谷	3296	刺客信条
洛谷	2601	对称正方形
洛谷	4323	独特的树叶
洛谷	4371	POC





练习题

洛谷 2375 动物园

洛谷 2463 Sandy的卡片

洛谷 3234 抄卡组

洛谷 2018 一木双棋

洛谷 3193 GT考试





练习题

洛谷	3294	背单词
洛谷	3879	阅读理解
洛谷	4283	异或粽子
洛谷	4735	最大异或和
洛谷	2292	L语言





练习题

洛谷	2414	阿里的打字机
洛谷	3796	AC自动加加强版
洛谷	4052	文本生成器
洛谷	5231	玄武密码
洛谷	3167	通配符匹配

